

# Chapter 11 — Orchestration

Delegating beyond one context window

Chapter 9 gave you one competent session. This is what to do when one is not enough — and how to tell when one was.



# Why this exists at all

- One session, one window. Chapter 1 said it is bounded; Chapter 5 said it degrades before it fills.
- A task that needs more reading than fits cannot be done by making the window bigger. It has to be split.
- Orchestration is that split, made explicit: several sessions, each with its own window, reporting back.

The unit of delegation is a context window, not a task. That reframing is most of the chapter.



# Plan mode at scale

- On a large change, the plan is the artefact you review — not the diff, which arrives too late and too long.
- Chapter 9 made the cost asymmetry at the scale of one change. At forty files it stops being an argument and becomes the only workable review point.
- A plan you cannot follow is a plan the model cannot execute either. Chapter 3's colleague test applies unchanged.



# Subagents

- A subagent is a fresh session with its own window, given one scoped task, returning a result rather than a transcript.
- Good for breadth: search several places at once, review several dimensions independently, sweep a directory.
- Bad for anything needing the judgement you are holding.

The rest of this slide is the instructor's practice, not documentation. A subagent reasoning from priors returns fluent, ungrounded output that reads exactly like verification. **Give it the evidence, not the question** — file paths, the actual numbers, the exact claim — and permission to answer "insufficient evidence".

Mechanism from the documentation; the framing above the line is mine.



# Hooks — lab standards, enforced

- A hook fires on an event: before a command runs, when a session starts, when a prompt is submitted.
- It is the difference between a convention written in a file and a convention that actually happens.
- Prose in an instruction file is a **request**. A hook is a **guarantee**.

Worked example available in this repository: a rule that a particular command must always carry a particular environment variable is not left to memory. It is rewritten automatically, and the rewrite is announced.



# Workflows, and the setting that runs them

- A workflow is orchestration written down: what fans out, what verifies, what synthesises — deterministic control flow around non-deterministic workers.
- It can be saved and re-run, which turns a good session into a group procedure.
- **Ultracode** is the session-scoped setting that pairs the highest reasoning effort with automatic workflow orchestration.
- Three ways in, as recorded: `/effort ultracode` in a session, `claude --effort ultracode` at launch (from v2.1.203), or the bare keyword to apply it to a single task.



# The three things people get wrong about ultracode

1. It is **session-scoped**. Put it in a persistent settings field and it is *silently ignored* — no error, no warning, and you believe it is on for weeks.
2. **Subagents auto-approve file edits**. The diff-review discipline from Chapter 9 does not hold inside a fan-out — that is the trade you are making. **The record does not say this is limited to workflows**, so assume it applies to any subagent until you have checked the current documentation yourself.
3. It carries a **significant cost premium**. On individual subscriptions this is where limits actually get hit. **No multiplier is published**, and this deck does not invent one — treat "significant" as the whole of what is known.

Demonstrate on **one directory**, not a repository. And do not leave the session in ultracode afterwards — the premium keeps applying to work that does not need it.



# Inspecting running work

- Long fan-outs are opaque by default. You need to see which agents are running, which have returned, and what they returned.
- Watch it once on something small. An orchestration you cannot inspect is an orchestration you cannot debug.

Version-fragile. The specific commands, flags and version floors here are correct as recorded and are re-checked in the week before the session.



# The judgement call

## Worth it

- The work genuinely exceeds one window
- Independent parts that do not need each other's results
- You want several independent looks at the same thing
- The task will be repeated, so the workflow amortises

## An expensive way to do something simple

- One file, one change
- Parts that must be done in order anyway
- You have not defined what "done" looks like
- You are orchestrating because it is impressive

Fan-out multiplies cost immediately and quality only sometimes. Ask what the second agent is for before you spawn it.



# Cost, picked up

- Chapter 7 priced one forward pass. Orchestration runs many, concurrently, at the highest effort setting.
- The premium is not a rounding error — though its size is unpublished — and the room is on individual subscriptions.
- Teach the usage check early, run heavy blocks before the afternoon, and pair when someone hits a limit.  
`TODO(cite)` — the exact command name is not in `references.md` ; confirm it against the documentation in delivery week rather than reading it off this slide.

---

Thread — cost. Chapter 13 closes it, with what a call is actually billed at.



## Where this leaves us

- Delegation whose unit is a context window.
- Hooks that turn a convention into a guarantee.
- One setting with three traps, all of which are silent.

Chapter 12 reaches outside the session entirely — and reopens the trust boundary Chapter 6 drew.

